

# INPUT/OUTPUT BUILT-IN FUNCTIONS

CSI910  
Computer Science I

## FIND A PARTNER: INPUT AND OUTPUT

- Let's simulate a program that reads numbers from the user (forever ... but we will do 6) and after each number report out the largest number seen to that point.
- Example set of numbers: 5, 6, 3, 2, 6, 8
- How would you do this in partners?

# COMPUTATIONAL THINKING: ALGORITHM

- Let's try an example with a partner – what is the overall algorithm?
- In pairs: One person is a sender, and one is a receiver
  1. Sender write a set of 6 numbers.
  2. One number at a time, Receiver asks “Can I have the next number?”, and the Sender gives it.
  3. Repeat the above line 6 times
  4. Sender asks “What was the highest number I gave you?”
  5. The Receiver reports what the highest number was.

## QUESTIONS TO ASK

1. What are we trying to achieve?
2. What do we know?
3. What information is missing?
4. Can we break it down into smaller parts?
5. What are the patterns?
6. Can we represent it differently?
7. Have we solved it before?

# QUESTIONS TO ASK

1. What are we trying to achieve?
  - We are taking in numbers and printing the largest seen so far
  - Do we need to track all of the numbers?
2. What do we know?
  - That the user can stop after the first or the millionth (or higher)
3. What information is missing?
  - How do we stop it? What does forever really mean?
  - We only have to report the highest!
4. Can we break it down into smaller parts?
  - In the slide above, I gave you an initial algorithm – what can you give a name the steps if they were on a computer?

## COMPUTATIONAL THINKING: ALGORITHM

- Decomposing the algorithm
  1. Sender create a set of 6 numbers.
    - Create by Sender
  2. One number at a time, Receiver asks “Can I have the next number?”
    - Output: (prompt) from Receiver to Sender
  3. Sender says the next number
    - Input from Sender to Receiver.
  4. Receiver records the number
    - Input by Receiver
  5. Sender asks “What was the highest number I gave you?”
    - Output: Sender to Receiver
  6. The Receiver reports what the highest number was.
    - Receiver compares new number to all other numbers
    - Output: Receiver to Sender
  7. Repeat forever ...

# WHAT ARE WE REALLY DOING HERE?

- How to build a program to read numbers from the user (forever) and after each number print out the largest number seen yet
  - We are not yet looking for python code – we are instead looking to use these skills

Decomposition

Abstraction

Pattern Recognition

Algorithm Design

Validation

# COMPUTATIONAL THINKING GROUP EXERCISE

- How to build a program to read numbers from the user (forever) and after each number print out the largest number seen yet
  - We are not yet looking for python code

## Decomposition

What are the essential tasks?

- Get input
- Convert input to int
- Keep track of largest
- Printing

## Abstraction

Require a variable to store the largest and the most recent (current)

## Pattern Recognition

A loop that runs forever each time ask for input potentially update the largest seen

## Algorithm Design

Initialize a variable  
Loop forever  
Ask for input  
Update the variable  
Print the variable

## Validation

simple cases  
negative numbers  
Invalid inputs?



# COMPUTATIONAL THINKING: ALGORITHM REFINEMENT

- Decomposing the algorithm
  1. Sender create a set of 6 numbers.
    - Create by Sender
  2. One number at a time, Receiver asks “Can I have the next number?”
    - Output: (prompt) from Receiver to Sender
  3. Sender says the next number
    - Input from Sender to Receiver.
  4. Receiver records the number
    - Input by Receiver
  5. Sender asks “What was the highest number I gave you?”
    - Output: Sender to Receiver
  6. The Receiver reports what the highest number was.
    - Receiver compares new number to all other numbers
    - Output: Receiver to Sender
  7. Repeat forever ...
- Now the next questions ...
- What are the patterns?
  - Questions are output asking for input
  - Statements are the input themselves
  - There is a second pattern:
    - We are constantly comparing different numbers - do we need to do that?
- Can we represent it differently?
  - The answer to the above question actually informs this – how do we avoid comparing different numbers?
- Have we solved it before?
  - Have you seen anything that lets you compare maximum values?

## KEEPING TRACK OF THE LARGEST?

- 2 Variables

- Most recently entered value
- Max ever seen (so far)

Give them names:  
`latest_value`  
`highest_value`

- How to update `highest_value` so it contains the maximum between itself and `latest_value`?
  - Can you come up with a single expression?

Given the problem do we need any other variables? i.e., do we need to keep track of anything else?

# TRANSLATE TO PSEUDOCODE

- Create a pseudocode algorithm for the above problem
  - Remember that you have seen some looping before
  - When you want some output you can write “PRINT” as your pseudocode
  - When you want some input from the user write “INPUT” and in <angle brackets> write what input you are waiting for

# PSEUDOCODE

- START
  - `highest_number = minus infinity` #so my first number is always higher
  - Repeat until INPUT is 'Quit'
    - PRINT "Please enter the next number:"
    - INPUT <Wait for next number>
    - SET latest\_number to INPUT
    - `Highest_number = maximum of highest_number and latest_number`
      - #Remember from last time we had a built-in called **max**
- END

## CODE EXAMPLES

- [https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/06-Decisions/CS1910\\_Decisions\\_Workbook.ipynb](https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/06-Decisions/CS1910_Decisions_Workbook.ipynb)

# PRINTING

- In Python the built-in `print` method can be used to print values to the console

```
print("hello")
```

```
hello = "Hi!"  
print(hello)
```

```
print(3)
```

Print automatically  
converts an int to  
string

```
print(str(3))
```

Convert the  
integer 3 to a  
string and  
then print it

```
print("hello" + 3)
```

Error can't add string and int

```
print("hello" + str(3))
```

"hello3"

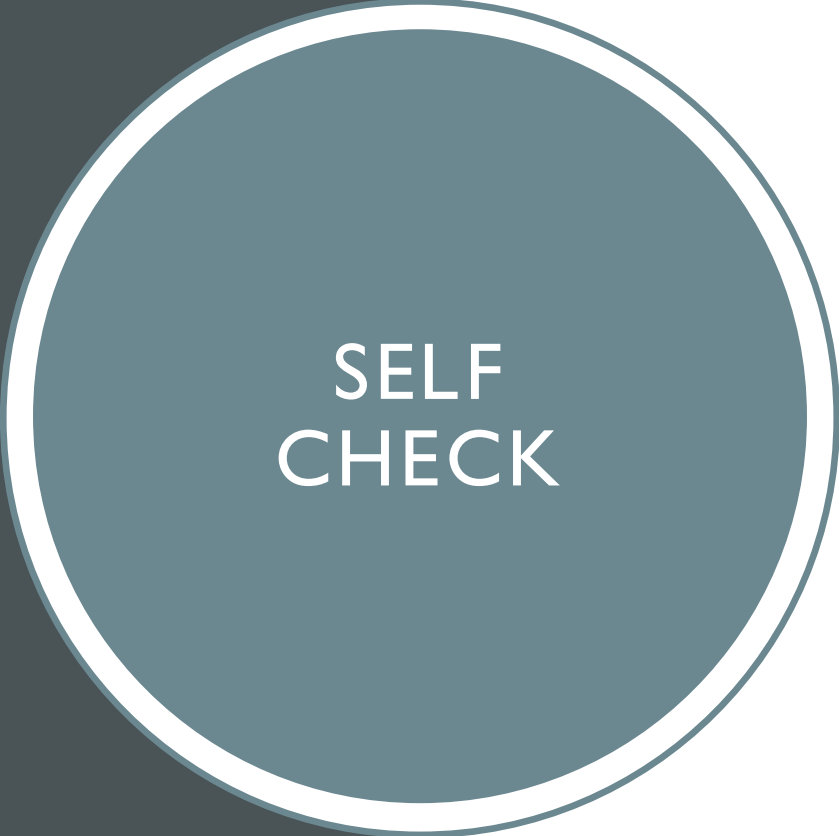
# ESCAPE SEQUENCES

- Certain characters are challenging to represent in a string
- For these we can represent the character using an escape sequence.
  - A special character combo that stands in for the challenging character
  - In Python we use a backslash `\` followed by a character to indicate an escape sequence
- Example: How to represent a newline within a string?
- `\n` represents a newline “`Hello\nWorld`” translates to `Hello`  
`World`

# EXAMPLE ESCAPE SEQUENCES

- **Cursor positioning escape sequences:**
  - `\b` = Backspace
  - `\t` = Tab
  - `\n` = Newline
  - `\r` = Carriage return
- **Character used in syntax escape sequences**
  - `\"` - produces a double quote
  - `\'` - produces a single quote
  - `\\` - produces a `\`

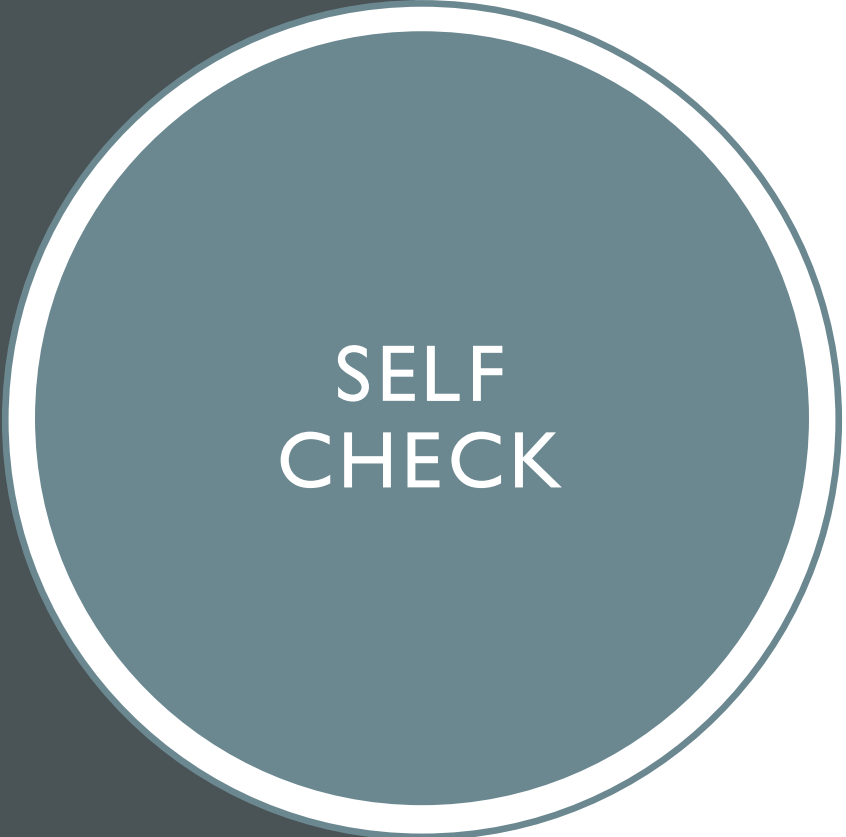




## SELF CHECK

- What is the output for the following?

```
print("\\"\\Hello\\n\\tWorld!\\\\"")
```



## SELF CHECK

- What is the output for the following?

```
print("\\"\\Hello\\n\\tWorld!\\\\"")
```

```
"\\Hello  
World!\""
```

# CLASS PARTICIPATION

On Moodle you will find:

**Class Participation – Review – Expressions**

# FORMATTED STRINGS

## F-STRINGS

- Allow you to embed variables and expressions into strings
- Preface the string with an `f` and then use curly braces `{ }` to indicate an expression or variable that should be inserted into the string
- Example

```
x = 10
y = 5
result_str = f"The sum of {x} and {y} is {x+y} or
what is stored in z which is {z}"
```

## FORMAT SPECIFIERS

- Control the presentation of values such as number of decimal places for floating point values or alignment

```
price = 123.4567
formatted_price = f"Price: ${price:.2f}" # Two decimal places
print(formatted_price)
```

```
number = 7
padded_number = f"Number: {number:03d}" # Pad with leading zeros to 3 digits
print(padded_number)
```

## SELF-CHECK

- Consider you have files named as follows:
- `file001.txt`, `file002.txt`, ... `file999.txt`
- Given an integer variable `file_num` (between 1 and 999) construct the appropriate file name (string) as per above.

## SELF-CHECK

- Consider you have files named as follows:
- `file001.txt`, `file002.txt`, ... `file999.txt`
- Given an integer variable `file_num` (between 1 and 999) construct the appropriate file name (string) as per above.

```
file_name = f"file{file_num:03d}.txt"
```

# INPUT

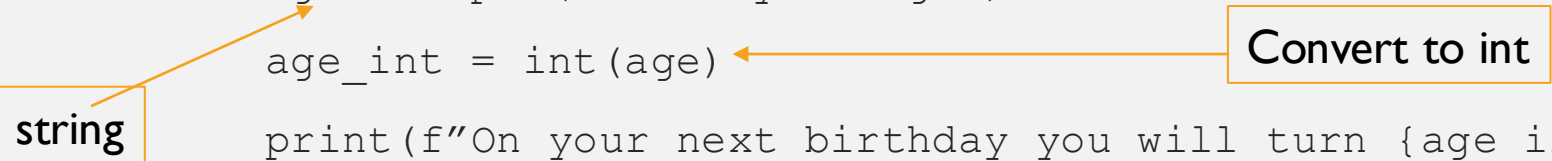
- User input from the keyboard can be accepted using the `input()` function
- The function accepts a prompt that is displayed to the console
  - `name = input("Enter your name:")`
- All input is returned as a string
- If numbers are intended as input they will have to be converted to integers or floats explicitly

```
age = input("Enter your age")
```

```
age_int = int(age)
```

```
print(f"On your next birthday you will turn {age_int + 1}")
```

string



Convert to int



## SELF CHECK

- What is the output?

```
age = input("Enter your age")  
print(f"On your next birthday you will turn {age + 1}")
```

## CONVERTING INPUT TO INT

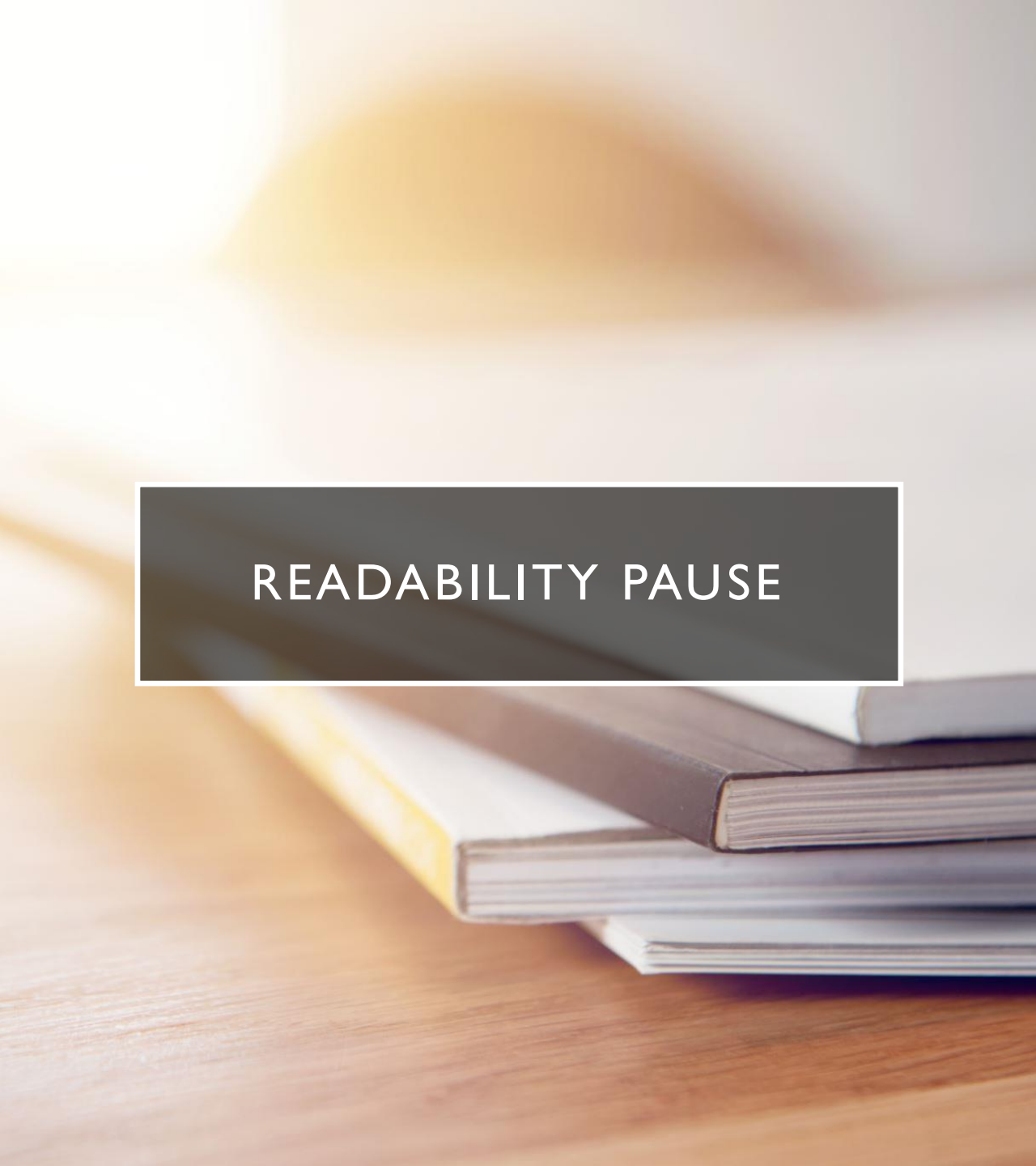
- Recall that input is read in as a String.
- We have to convert it to an integer.
  - “3” is not 3
- `current_value_str = input("Enter a value")`
- `current_value = int (current_value_str)`

## FIND A PARTNER

- Write a program to ask the user to enter 3 floating point numbers and then print the average.
- The user should enter them one after the other (not all on the same input)
- Make use of the += (add and assign operator) as part of your solution.

# VOLUNTEERS

- Some volunteers to share their solutions to the average of 3 numbers problem
  - On the board or with a share to google colab



## READABILITY PAUSE

- It is important to write code that is easy to understand, interpret and read

# VARIABLE NAMES

- We have mentioned a few rules about python names w.r.t variables
- But there are also best practices
- 1. Choose meaningful descriptive names
  - Avoid single letters most of the time,
  - Use pot\_hole\_case (aka snake\_case) when using multiple word variable names
- 2. Be consistent – choose a style and stick to it

# WHITE SPACE

Surround binary operators with a single white space character

```
# white space with expressions  
# use single spaces with binary operators  
# be consistent
```

```
#yes
```

```
student_id = 29  
student_id += 1  
student_id = student_id + 1
```

```
#no
```

```
student_id= 29  
student_id+=1  
student_id = student_id + 1
```

## WHITE SPACE

Avoid spaces with function  
call expressions

*# white space with function calls*  
*# do not put a space between parenthesis*

*#yes*

```
print("hello")
```

*#no*

```
print ("hello")
```

```
print( "hello" )
```

```
print ( "hello" )
```



# COMMENTS

- Avoid using comments to support poorly written code

```
# Set a variable pi to 3.14159
pi = 3.14159
# Set a variable r to 7
r = 7
```

← The comments provides  
no added benefit

- Document the context of the program
- Help other programs understanding programming choices
  - Not a commentary on what lines of code do
  - Well written code should be self-documenting (good choice of variables, consistent naming and whitespace clear and concise)

# FUNCTIONS

- Last day we introduced function expressions
- A function call returns a value in the place of the call
  - It might be `None` (a special value representing nothing)
- Functions are a form of abstraction
  - Generalize a program or procedure based on inputs (called arguments)
- In terms of program flow so far we have programs that "flow" from the first statement (top) to the bottom
- When a function is called the program executes the code belonging to the function before continuing "down" to the next code in the program
- Some functions accept arguments that are passed to them
- E.g `max (5, 10)`

Arguments passed to the max function

# PROGRAM FLOW EXAMPLE

This is our own  
version of the built-in  
abs

We'll write our own  
Functions later in the  
course

First line of program  
execution

Direction of program flow

```
def abs(value):  
    if value < 0:  
        return -value  
    return value  
  
x = input("enter an integer value")  
x = int(x)  
  
abs_x = abs(x)  
  
print(f"abs of {x} is {abs_x}")
```

A function must be defined before  
it is used

This is why we import math to use  
math functions

Built-in functions (like abs) do not  
need to be imported (they are  
already visible)

Each line in this program calls a  
method. The program flow will jump  
to that method and execute its code  
before continuing

## MORE BUILT-INS

| Name | Number of Arguments | Description and Example                                                                                                                                              |
|------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| max  | $\geq 2$            | Returns the maximum value of all arguments. Accepts any number of arguments.<br><pre>&gt;&gt;&gt; largest = max(10, 15, 42, 19) &gt;&gt;&gt; print(largest) 42</pre> |
| min  | $\geq 2$            | Works like max but returns the minimum value of all arguments. Accepts any number of arguments.                                                                      |
| abs  | 1                   | Returns the absolute value of a number.<br><pre>&gt;&gt;&gt; print(abs(-18)) 18</pre>                                                                                |

## MORE BUILT-INS

|       |                              |                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| round | 1 (required)<br>1 (optional) | <p>Returns a floating point number that is a rounded version of the specified number (required), with the specified number of decimals (optional). The default number of decimals is 0, meaning that the function will return the nearest integer if only the required parameter is passed.</p> <pre>&gt;&gt;&gt; print(round(18.234))<br/>18<br/>&gt;&gt;&gt; print(round(18.274, 1))<br/>18.3</pre> |
| pow   | 2                            | <p>Calling pow(x,y) returns the value of x raised to the power of y.</p> <pre>&gt;&gt;&gt; print(pow(7,2))<br/>49<br/>&gt;&gt;&gt; print(pow(5.0, 1/2))<br/>2.23606797749979</pre>                                                                                                                                                                                                                    |
| type  | 1                            | <p>Returns the data type of its argument.</p> <pre>&gt;&gt;&gt; type(5)<br/>&lt;class 'int'&gt;<br/>&gt;&gt;&gt; type(2.3)<br/>&lt;class 'float'&gt;<br/>&gt;&gt;&gt; type("foo")<br/>&lt;class 'str'&gt;</pre>                                                                                                                                                                                       |

## MORE BUILT-INS

| Name               | Number of Arguments | Description and Example                                                                                                                                                                                                                                                                                                                                           |
|--------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int</code>   | 1                   | <p>Converts the argument to the integer data type (if possible) and returns the result. Strings can be converted if they contain only digits 0–9 and possibly a decimal point.</p> <pre>&gt;&gt;&gt; int(42.0) 42 &gt;&gt;&gt; int("42") 42</pre>                                                                                                                 |
| <code>float</code> | 1                   | <p>Similar to <code>int</code>; converts the argument to the float-point data type (if possible) and returns the result.</p>                                                                                                                                                                                                                                      |
| <code>str</code>   | 1                   | <p>Similar to <code>int</code>; converts the argument to the string data type and returns the result.</p>                                                                                                                                                                                                                                                         |
| <code>input</code> | 1 (optional)        | <p>This function may optionally be given a string argument. If given, the argument is printed as a prompt, then the function waits for the user to enter a string and press the enter key. The function returns the text entered. Yes, this is the same function we used for console input in Section 3.4.2!</p> <pre>x = input("Please enter your name: ")</pre> |

## SELF-CHECK

- What is the result?

```
float(str(f"{max(5,min(5,max(5,1)))}.25")) + int(5.25)
```

# EUCLIDEAN DISTANCE

- Suppose we have 2 points  $(x_1, y_1)$   $(x_2, y_2)$  we want to output the distance between the 2 points where distance is given by the below formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



## TRY AT HOME

- Prompt the user to enter their first name and then last name, output whichever has more letters in it. Hint use the `len` built-in function
- Create an adding machine to read 2 numbers from the user and then output their sum
- Change the Euclidean distance program to instead use Manhattan Distance
  - [https://simple.wikipedia.org/wiki/Manhattan\\_distance](https://simple.wikipedia.org/wiki/Manhattan_distance)

# TRY AT HOME

Build a Mad-Libs program, Prompt the user to input each of the blanks in the story to the right. After they have entered all the blanks output the story with the blanks filled in with their answers.

Read the output, hilarity ensues.

## PIZZA PIZZA



Pizza was invented by a \_\_\_\_\_  
(adjective) (nationality)

chef named \_\_\_\_\_. To make a pizza, you need  
(person)

to take a lump of \_\_\_\_\_, and make a thin, round  
(noun)

\_\_\_\_\_. Then you cover it with  
(adjective) (noun)

\_\_\_\_\_ sauce, \_\_\_\_\_ cheese, and fresh  
(adjective) (adjective)

chopped \_\_\_\_\_. Next you have to bake it in a very  
(plural noun)

hot \_\_\_\_\_. When it is done, cut it into \_\_\_\_\_  
(noun) (number)

\_\_\_\_\_. Some kids like \_\_\_\_\_ pizza the  
(shapes) (food)

best, but my favorite is the \_\_\_\_\_ pizza. If I could, I  
(food)

would eat pizza \_\_\_\_\_ times a day!  
(number)



## FROM LAST TRY AT HOME: HOTEL ROOM QUESTION HINTS

- Sum of the individual digits in a 3 digit number
- How to get the first digit (ones digit)?
- How to access the second digit?